چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۶

کار پس‌زمینه — بخش ۲: سرویس‌ها، foreground serviceها، alarmها و push



اسماعیل مرادی

@esmaeeil-moradi700

چرا بخش دوم در ۲۰۲۶ مهم است؟

بخش ۱ دربارهٔ کار قابل‌تعویق بود. بخش ۲ دربارهٔ بقیه است: کاری که کاربر **تماشایش می‌کند**، کاری که باید **سر یک زمان مشخص** انجام شود، و کاری که **سرور** شما شروعش می‌کند. اندروید بیشترین تغییر را همین‌جا داشته — و جواب‌های قدیمی همین‌جا با targetSdk جدید crash می‌کنند:

- سرویس‌ها هنوز بلوک سازنده‌اند — اما سرویس **پس‌زمینه** دیگر تقریباً وجود ندارد
- هر foreground service به یک **نوع** (type) و مجوز متناظرش نیاز دارد (اندروید ۱۴)
- شروع foreground service **از پس‌زمینه** تقریباً ممنوع است (اندروید ۱۲)
- exact alarm به **مجوزی** نیاز دارد که کاربر می‌تواند ردش کند
- انتقال‌های حجیمی که کاربر شروع می‌کند، نوع job مخصوص خودشان را دارند: **UIDT** (اندروید ۱۴)

سطح	سؤال‌ها
جونیور	انواع سرویس، started service، bound service
مید	foreground service، نوع FGS، شروع از پس‌زمینه، alarmها
سینیور	JobScheduler و UIDT، boot و زمان‌بندی دوباره، FCM با اولویت بالا
آرکیتکت	انتخاب ابزار درست، مهاجرت به targetSdk 35 به بالا، یادآورهای قابل‌اعتماد



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: چه چیزی کار را زنده نگه می‌دارد؟

Mechanism	What keeps the work alive
viewModelScope	the screen
Foreground service	a visible notification
UIDT job	user action + notification
WorkManager / job	the system scheduler
Alarm	a moment in time
High-priority FCM	your server

- هر سازوکار یک **قرارداد** با سیستم است — و هر کدام هزینه‌ای دارد (یک notification، یک مجوز، یک quota)
- یک سرویس پس‌زمینهٔ ساده هیچ چیزی را زنده نگه نمی‌دارد: سیستم کمی بعد از رفتن اپ از پیش‌زمینه متوقفش می‌کند
- **ضعیف‌ترین** سازوکاری را انتخاب کنید که هنوز نیاز را برآورده می‌کند

قاعدهٔ واحد پشت همهٔ جواب‌های امروز

اگر نه کاربر از کار شما خبر دارد و نه زمان‌بند سیستم، فرض کنید کشته می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

انواع سرویس، started service، bound service



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۱. سرویس‌ها در ۲۰۲۶: سه نوع

نوع	طول عمر	واقعیت امروز
Started — `startService()`	تا `stopSelf()` یا `stopService()`	از پس‌زمینه قابل شروع نیست (اندروید ۸ به بعد)؛ چند دقیقه بعد از رفتن اپ از پیش‌زمینه متوقف می‌شود
Bound — `bindService()`	تا وقتی آخرین کلاینت unbind کند	برای APIهای داخل اپ یا بین processها خوب است؛ راهی برای زنده ماندن نیست
Foreground — `startForegroundService()` و بعد `startForeground()`	تا وقتی متوقف شود؛ فقط در فشار شدید حافظه کشته می‌شود	notification، نوع و مجوز لازم است

کلاس `IntentService` که در API 30 منسوخ شد، و `JobIntentService`، دیگر در کد مدرن جایی ندارند — از WorkManager استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«سرویس در پس‌زمینه اجرا می‌شود، پس روی ترد اصلی نیست.» — `onCreate()` و `onStartCommand()` و `onBind()` همه روی **ترد اصلی** اجرا می‌شوند. Service کامپوننتی است که سیستم آن را می‌شناسد، نه یک ترد. block کردنش ANR می‌دهد.

تلهٔ مصاحبه

«foreground service حتی وقتی اپ کشته شود به کارش ادامه می‌دهد.» — نسخهٔ پارسال این را گفته بود. سرویس **داخل process شما** زندگی می‌کند: اگر process بمیرد، سرویس هم با آن می‌میرد. foreground بودن احتمال کشته شدن را خیلی کم می‌کند — نه صفر.

سؤال تکمیلی

«پس چرا هنوز سرویس می‌نویسیم؟» — برای کاری که کاربر فعالانه از آن خبر دارد (پخش، مسیریابی، تماس) و برای ارائهٔ API به کامپوننت‌ها یا اپ‌های دیگر (bound service). هر چیز قابل‌تعویق به WorkManager می‌رود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. یک started service، به شکل درست

```
class ExportService : Service() {
    private val scope =
        CoroutineScope(SupervisorJob() + Dispatchers.Default)

    override fun onStartCommand(
        intent: Intent?, flags: Int, startId: Int,
    ): Int {
        scope.launch {
            exporter.export()
            stopSelf(startId)
        }
        return START_NOT_STICKY
    }

    override fun onDestroy() {
        scope.cancel()
        super.onDestroy()
    }

    override fun onBind(intent: Intent?): IBinder? = null
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. مقدار بازگشتی و توقف

خروجی `onStartCommand()`	اگر process کشته شود
`START_NOT_STICKY`	دوباره ساخته نمی‌شود — پیش‌فرض درست
`START_STICKY`	بعداً دوباره ساخته می‌شود، با intent **null**
`START_REDELIVER_INTENT`	دوباره ساخته می‌شود و آخرین intent دوباره تحویل داده می‌شود

- `stopSelf(startId)` فقط وقتی متوقف می‌کند که درخواست شروع جدیدتری نرسیده باشد؛ `stopSelf()` در هر حال متوقف می‌کند — و ممکن است کار یک `startService()` بعدی را هم بکشد
- scope خود را در `onDestroy()` لغو کنید، وگرنه coroutineها بیشتر از سرویس عمر می‌کنند

تلهٔ مصاحبه

«*`START_STICKY` سرویس مرا زنده نگه می‌دارد.*» — این فقط یک راهنمایی برای راه‌اندازی دوباره است، نه زنده نگه داشتن: راه‌اندازی دوباره ممکن است دیر انجام شود، intent برابر `null` است، و همهٔ محدودیت‌های پس‌زمینه همچنان برقرارند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. bound serviceها

```
class PlayerService : Service() {
    inner class LocalBinder : Binder() {
        fun service(): PlayerService = this@PlayerService
    }
    private val binder = LocalBinder()
    override fun onBind(intent: Intent): IBinder = binder
}

// Client: bind in onStart(), unbind in onStop()
val connection = object : ServiceConnection {
    override fun onServiceConnected(
        name: ComponentName, binder: IBinder,
    ) {
        player = (binder as PlayerService.LocalBinder).service()
    }
    override fun onServiceDisconnected(name: ComponentName) {
        player = null
    }
}
```

- `bindService(intent, connection, Context.BIND_AUTO_CREATE)` در صورت نیاز سرویس را می‌سازد؛ وقتی آخرین کلاینت unbind کند از بین می‌رود (مگر اینکه start هم شده باشد)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. جزئیات binder که اهمیت دارند

- `Binder` محلی فقط **در همان process** کار می‌کند. بین processها: `Messenger` (یک صف) یا **AIDL** (فراخوانی‌های هم‌زمان)
- فراخوانی‌های AIDL از process دیگر روی **thread pool مربوط به binder** می‌رسند، نه ترد اصلی — پیاده‌سازی باید thread-safe باشد
- اپ‌های رسانه‌ای: `MediaSessionService` در Media3 کار bind و foreground service را برایتان انجام می‌دهد

تلهٔ مصاحبه

«*`onServiceDisconnected()` وقتی unbind می‌کنم صدا زده می‌شود.*» — نه. فقط وقتی اتصال **به‌طور غیرمنتظره قطع شود** صدا زده می‌شود (process سرویس crash کرده یا کشته شده). وقتی `unbindService()` را صدا می‌زنید، ارجاع را خودتان پاک کنید.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

foreground service، نوع FGS، شروع از پس‌زمینه، alarmها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. یک foreground service در ۲۰۲۶

```
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission
    android:name="android.permission.FOREGROUND_SERVICE_LOCATION" />

<service
    android:name=".tracking.RunTrackingService"
    android:foregroundServiceType="location"
    android:exported="false" />
```

```
class RunTrackingService : LifecycleService() {
    override fun onStartCommand(
        intent: Intent?, flags: Int, startId: Int,
    ): Int {
        super.onStartCommand(intent, flags, startId)
        ServiceCompat.startForeground(
            this, NOTIFICATION_ID, trackingNotification(),
            ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION,
        )
        lifecycleScope.launch { tracker.track() }
        return START_NOT_STICKY
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. چک‌لیست

۱. مجوز `FOREGROUND_SERVICE` (اندروید ۹ به بعد) **و** مجوز مخصوص نوع، مثلاً `FOREGROUND_SERVICE_LOCATION` (اندروید ۱۴ به بعد)

۲. ویژگی `android:foregroundServiceType` در manifest، و همان نوع در فراخوانی `startForeground()`

۳. یک notification روی یک channel. بدون `POST_NOTIFICATIONS` (اندروید ۱۳ به بعد) سرویس باز هم اجرا می‌شود، اما notification آن فقط در Task Manager دیده می‌شود

۴. بعد از `startForegroundService()`، متد `startForeground()` را در همان پنجرهٔ چندثانیه‌ای مستند صدا بزنید — وگرنه اپ ANR یا crash می‌شود

۵. پیش‌نیازهای هر نوع هنگام شروع چک می‌شوند: نوع `location` به مجوز مکانِ اعطاشده نیاز دارد، وگرنه `startForeground()` خطای `SecurityException` می‌دهد

۶. توقف با `ServiceCompat.stopForeground(this, STOP_FOREGROUND_REMOVE)` و `stopSelf()` — متد `stopForeground(true)` منسوخ شده

تلهٔ مصاحبه

«`FOREGROUND_SERVICE_CAPABILITY` *را اعلام کن؛ نوع سرویس از اندروید ۱۲ اجباری است.*» — هر دو اشتباه است (نسخهٔ پارسال). چنین مجوزی **وجود ندارد**. نوع سرویس برای اپ‌هایی که **اندروید ۱۴** را هدف می‌گیرند اجباری شد؛ قبل از آن فقط دسترسی به مکان (اندروید ۱۰) و دوربین و میکروفون (اندروید ۱۱) نوع لازم داشت.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. انتخاب نوع FGS

نوع	کاربرد رایج	نیاز اضافه
`mediaPlayback`	صدا یا ویدیویی که در پس‌زمینه ادامه دارد	`FOREGROUND_SERVICE_MEDIA_PLAYBACK`
`location`	مسیریابی، ثبت دویدن	مجوز خودش، به‌علاوهٔ مجوز مکانِ اعطاشده
`connectedDevice`	ساعت هوشمند، لوازم بلوتوثی	مجوز خودش، به‌علاوهٔ مثلاً مجوز بلوتوث
`phoneCall`	یک تماس در جریان	مجوز خودش، به‌علاوهٔ نقش یا مجوز تماس
`dataSync`	آپلود، دانلود، پشتیبان‌گیری	`FOREGROUND_SERVICE_DATA_SYNC` — با سقف زمانی (اندروید ۱۵)
`shortService`	تمام کردن یک کار کوتاه و حیاتی	بدون مجوز نوع؛ حدود ۳ دقیقه، بعد `onTimeout()`
`specialUse`	هیچ نوع دیگری جور نیست	مجوز خودش، به‌علاوهٔ یک `<property>` در manifest که کاربرد را توضیح دهد

- اپ‌هایی که اندروید ۱۴ یا بالاتر را هدف می‌گیرند باید **نوع FGSهای خود را در Play Console اعلام کنند**، همراه با توجیه — نوع اشتباه یعنی رد شدن در بررسی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. سقف‌های زمانی اندروید ۱۵

ANDROID 15 (API 35)

برای اپ‌هایی که API 35 را هدف می‌گیرند، نوع `dataSync` (و نوع جدید `mediaProcessing`) می‌تواند **۶ ساعت در هر بازهٔ ۲۴ ساعته** اجرا شود. با رسیدن به سقف، سیستم `Service.onTimeout(startId, fgsType)` را صدا می‌زند؛ ظرف چند ثانیه متوقف کنید وگرنه اپ crash می‌کند. وقتی کاربر اپ را به پیش‌زمینه بیاورد، شمارنده از نو شروع می‌شود.

```
override fun onTimeout(startId: Int, fgsType: Int) {
    checkpointStore.save(currentProgress)
    stopSelf(startId)
}
```

تلهٔ مصاحبه

«`dataSync` *نوع پیش‌فرض برای هر کار پس‌زمینه است.*» — اپ‌ها دقیقاً همین‌طور علامت‌گذاری شدند. sync قابل‌تعویق به WorkManager می‌رود. انتقال حجیمی که کاربر شروع کرده به یک job از نوع UIDT (سؤال ۸). `dataSync` برای حالت‌های بینابین است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. شروع FGS از پس‌زمینه

از **اندروید ۱۲** (targetSdk 31 به بالا)، اپی که در پس‌زمینه است نمی‌تواند foreground service شروع کند — مگر در استثناهای مستند، از جمله:

- اپ **همین الان قابل‌مشاهده** بوده (چند لحظه پیش یک activity روی صفحه بود)
- کاربر روی **notification، ویجت یا bubble** اپ شما زده
- یک **exact alarm** برای کاری که کاربر خواسته اجرا می‌شود
- یک پیام **FCM با اولویت بالا** می‌رسد
- دریافت `BOOT_COMPLETED` یا `LOCKED_BOOT_COMPLETED` یا `MY_PACKAGE_REPLACED` (با محدودیت‌های اندروید ۱۵ — سؤال ۹)
- کاربر **battery optimization** را برای اپ شما خاموش کرده

```
try {
    ContextCompat.startForegroundService(context, intent)
} catch (e: IllegalStateException) {
    val notAllowed = Build.VERSION.SDK_INT >= 31 &&
        e is ForegroundServiceStartNotAllowedException
    if (notAllowed) scheduler.enqueueExpedited(task)
    else throw e
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. دو تله در یک قاعده

تلهٔ مصاحبه

«*من مستثنا هستم، پس FGS من می‌تواند از دوربین، میکروفون یا مکان استفاده کند.*» — اجازهٔ شروع یک بررسی است؛ **دسترسی while-in-use** بررسی دیگری است (اندروید ۱۱). FGSای که از پس‌زمینه شروع شده نمی‌تواند از مکان، دوربین یا میکروفون استفاده کند، مگر اینکه اپ دسترسی لازم را داشته باشد یا استثنای مشخصی برقرار باشد.

تلهٔ مصاحبه

«*یک push با اولویت بالا بفرست تا هر FGSای شروع شود.*» — FCM پیام‌های اولویت‌بالایی را که به notification قابل‌مشاهده برای کاربر منجر نمی‌شوند **کم‌اولویت می‌کند** (سؤال ۱۰). طراحی مبتنی بر pushی که بی‌صدا سرویس راه می‌اندازد، در production می‌شکند.

سؤال تکمیلی

«*جایگزین چه باشد؟*» — WorkManager از نوع expedited برای کار کوتاه و فوری؛ در غیر این صورت یک notification نشان دهید که کاربر با آن اپ را جلو بیاورد و FGS از همان‌جا شروع شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. alarmها: غیردقیق، دقیق، alarm clock

API	دقت	مجوز exact alarm (اندروید ۱۲ به بعد)
`set()`، `setWindow()`، `setInexactRepeating()`	غیردقیق، دسته‌بندی‌شده	لازم نیست
`setAndAllowWhileIdle()`	غیردقیق، در Doze هم اجرا می‌شود (با محدودیت نرخ)	لازم نیست
`setExact()`، `setExactAndAllowWhileIdle()`	دقیق	**لازم است**
`setAlarmClock()`	دقیق، به کاربر نشان داده می‌شود، دستگاه را از Doze بیدار می‌کند	**لازم است**

مجوز	از	برای	اعطا
`SCHEDULE_EXACT_ALARM`	اندروید ۱۲	اپ‌هایی که نیاز واقعی دارند	کاربر می‌تواند لغوش کند؛ در اندروید ۱۴ به بعد برای بیشتر نصب‌های جدید **به‌طور پیش‌فرض رد شده**
`USE_EXACT_ALARM`	اندروید ۱۳	فقط اپ‌های ساعت زنگ‌دار، تایمر و تقویم — طبق سیاست Play	هنگام نصب؛ قابل لغو نیست



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. زمان‌بندی امن یک exact alarm

```
val canExact = Build.VERSION.SDK_INT < Build.VERSION_CODES.S ||
    alarmManager.canScheduleExactAlarms()

if (canExact) {
    alarmManager.setExactAndAllowWhileIdle(
        AlarmManager.RTC_WAKEUP, triggerAtMillis, pendingIntent,
    )
} else {
    alarmManager.setAndAllowWhileIdle(
        AlarmManager.RTC_WAKEUP, triggerAtMillis, pendingIntent,
    )
    // Offer Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM
}
```

- صدا زدن API دقیق بدون مجوز، `SecurityException` می‌دهد
- به `ACTION_SCHEDULE_EXACT_ALARM_PERMISSION_STATE_CHANGED` گوش دهید تا وقتی مجوز داده شد alarmها را دقیق کنید؛ اگر کاربر مجوز را لغو کند، اپ متوقف و exact alarmهای آینده لغو می‌شوند

تلهٔ مصاحبه

exact alarm برای sync دوره‌ای. دستگاه را طبق برنامهٔ شما بیدار می‌کند، باتری را خالی می‌کند و با سیاست exact alarm در Play جور نیست. جای sync در WorkManager است؛ alarm برای لحظه‌هایی است که برای **کاربر** مهم‌اند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

JobScheduler و UIDT، boot و زمان‌بندی دوباره، FCM با اولویت بالا



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. استفادهٔ مستقیم از JobScheduler

```
class ReindexJobService : JobService() {
    private val scope = MainScope()

    override fun onStartJob(params: JobParameters): Boolean {
        scope.launch {
            indexer.reindex()
            jobFinished(params, false)
        }
        return true // work continues asynchronously
    }

    override fun onStopJob(params: JobParameters): Boolean {
        scope.coroutineContext.cancelChildren()
        return true // reschedule
    }
}
```

- سرویس را در manifest با مجوز `BIND_JOB_SERVICE` اعلام کنید
- `setPersisted(true)` job را بعد از ری‌استارت نگه می‌دارد (به `RECEIVE_BOOT_COMPLETED` نیاز دارد)
- پیش‌فرض WorkManager است؛ فقط برای قابلیتی که WorkManager ندارد، مثل UIDT، مستقیم سراغ JobScheduler بروید

تلهٔ مصاحبه

«*استفادهٔ مستقیم از JobScheduler پایداری را از بین می‌برد.*» — ادعای نسخهٔ پارسال؛ `setPersisted(true)` از ری‌استارت عبور می‌کند. آنچه از دست می‌دهید امکانات WorkManager است: زنجیره، unique work، observability و ابزار تست.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. jobهای انتقال داده به درخواست کاربر (UIDT)

```
val job = JobInfo.Builder(JOB_ID, component)
    .setUserInitiated(true)
    .setRequiredNetworkType(JobInfo.NETWORK_TYPE_ANY)
    .setEstimatedNetworkBytes(downloadBytes, 0L)
    .build()
jobScheduler.schedule(job)

// In the JobService
override fun onStartJob(params: JobParameters): Boolean {
    setNotification(
        params, NOTIFICATION_ID, progressNotification(),
        JobService.JOB_END_NOTIFICATION_POLICY_REMOVE,
    )
    startTransfer(params)
    return true
}
```

- اندروید ۱۴ (API 34)؛ مجوز `RUN_USER_INITIATED_JOBS` ؛ باید **وقتی اپ قابل‌مشاهده است زمان‌بندی شود** (یا در چند وضعیت مجاز دیگر) و constraint شبکه داشته باشد
- وقتی شرایط اجازه دهد می‌تواند مدت طولانی اجرا شود؛ در تمام این مدت یک notification نشان می‌دهد
- برای «این فصل را دانلود کن» و «این ۲۰۰ عکس را آپلود کن» ساخته شده. در زمان نوشتن این متن WorkManager آن را ارائه نمی‌دهد — از JobScheduler استفاده کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. boot و زمان‌بندی دوباره

```
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        // Main thread, short budget: hand off to WorkManager
        val request = OneTimeWorkRequestBuilder<RescheduleWorker>()
            .build()
        WorkManager.getInstance(context).enqueueUniqueWork(
            "reschedule-reminders",
            ExistingWorkPolicy.KEEP,
            request,
        )
    }
}
```

- در manifest: مجوز `RECEIVE_BOOT_COMPLETED` و یک receiver با intent filter برای `BOOT_COMPLETED`؛ مقدار `exported="false"` هم کار می‌کند — سیستم باز هم آن را تحویل می‌دهد

زمان‌بندی با	بعد از ری‌استارت	بعد از force stop
WorkManager	خودکار دوباره زمان‌بندی می‌شود	در اجرای بعدی اپ دوباره زمان‌بندی می‌شود
JobScheduler با `setPersisted(true)`	حفظ می‌شود	لغو می‌شود
AlarmManager	**پاک می‌شود** — در boot دوباره زمان‌بندی کنید	پاک می‌شود
foreground service	از بین رفته	از بین رفته



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تله‌های boot

تلهٔ مصاحبه

«*boot receiver من بلافاصله بعد از نصب کار می‌کند.*» — اپ تازه نصب‌شده تا وقتی کاربر یک بار بازش نکند در **stopped state** است؛ هیچ broadcastی نمی‌گیرد، از جمله `BOOT_COMPLETED` . بعد از force stop هم همین‌طور است.

تلهٔ مصاحبه

کار سنگین در `onReceive()` . روی ترد اصلی و با بودجهٔ زمانی کوتاه اجرا می‌شود. از `goAsync()` فقط برای چند ثانیه کار async استفاده کنید؛ هر چیز طولانی‌تر به WorkManager برود.

ANDROID 15 (API 35)

برای اپ‌هایی که API 35 را هدف می‌گیرند، receiver مربوط به `BOOT_COMPLETED` **نمی‌تواند** foreground serviceهایی از نوع `dataSync` ، `camera` ، `mediaPlayback` ، `phoneCall` ، `mediaProjection` یا `microphone` شروع کند — خطای `ForegroundServiceStartNotAllowedException` می‌دهد.

- حالت **Direct Boot**: پیام `BOOT_COMPLETED` بعد از اولین باز کردن قفل می‌رسد. ساعت زنگ‌داری که باید قبل از آن زنگ بزند از `LOCKED_BOOT_COMPLETED` ، `android:directBootAware` و حافظهٔ device-protected استفاده می‌کند — دیتابیس Room معمولی شما هنوز در دسترس نیست



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. پیام‌های FCM با اولویت بالا

```
class PushService : FirebaseMessagingService() {
    override fun onMessageReceived(message: RemoteMessage) {
        when (message.data["type"]) {
            "chat" -> notifier.showMessage(message.data)
            "sync" -> syncScheduler.requestSync()
        }
    }

    override fun onNewToken(token: String) {
        tokenRegistrar.register(token) // enqueues work
    }
}
```

	اولویت معمولی	اولویت بالا
دستگاه در Doze	در پنجرهٔ نگهداری بعدی تحویل می‌شود	همین حالا تحویل می‌شود؛ دستگاه را بیدار می‌کند
اپ چه کاری می‌تواند بکند	پردازش کوتاه	یک allowlist کوتاه: شبکه، expedited work، استثنای شروع FGS
مناسب برای	اشارهٔ sync، به‌روزرسانی غیرفوری	چیزی که کاربر **می‌بیند**: پیام چت، تماس، هشدار



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. چه چیزی در production می‌شکند

تلهٔ مصاحبه

اولویت بالا برای syncهای بی‌صدا. اگر پیام‌های اولویت‌بالا به notification قابل‌مشاهده منجر نشوند، FCM ممکن است آن‌ها را به اولویت معمولی **تنزل دهد**. مقایسهٔ `RemoteMessage.priority` با `originalPriority` نشان می‌دهد این اتفاق کی افتاده.

تلهٔ مصاحبه

کار طولانی در `onMessageReceived()`. پنجرهٔ زمانی کوتاه است — WorkManager از نوع expedited را enqueue کنید و برگردید.

- پیام‌های **notification** وقتی اپ در پس‌زمینه است مستقیم به system tray می‌روند و `onMessageReceived()` صدا زده نمی‌شود. وقتی اپ باید منطقی اجرا کند، از پیام‌های **data** استفاده کنید
- اپ‌هایی که force stop یا **hibernate** شده‌اند (ماه‌ها استفاده نشده‌اند) اصلاً پیامی نمی‌گیرند — حتی با اولویت بالا

سؤال تکمیلی

«push یا polling؟» — push برای گفتن «چیزی عوض شده»، و sync دوره‌ای با WorkManager به‌عنوان تور ایمنی (روز ۱۵، سؤال ۱۲).



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

انتخاب ابزار درست، مهاجرت به targetSdk 35 به بالا، یادآورهای قابل‌اعتماد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. انتخاب ابزار درست

```
Must it continue after the user leaves the screen?
├─ No  → coroutine in viewModelScope
└─ Yes → Must it happen at an exact time?
         ├─ Yes → exact alarm → notify or enqueue work
         └─ No  → Did the user start it and is watching?
                  ├─ Data transfer → UIDT job
                  ├─ Ongoing       → foreground service
                  └─ No            → WorkManager
                                     (expedited if urgent)
```

- از بالا شروع کنید و روی **اولین** ابزاری که نیاز را برآورده می‌کند بایستید — هر پله پایین‌تر هزینهٔ بیشتری در باتری، مجوز یا بررسی Play دارد
- سرور می‌تواند تقریباً هر شاخه‌ای را شروع کند: پیام FCM می‌تواند به notification، به expedited work یا به یک FGS مستثنا برسد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. سناریوهای محبوب مصاحبه‌گرها

سناریو	ابزار
به‌روز کردن یک لیست تا وقتی صفحه باز است	coroutine در `viewModelScope`
فرستادن پیام چت، حتی اگر کاربر برود	WorkManager از نوع expedited
پشتیبان‌گیری شبانه روی Wi-Fi و در حال شارژ	WorkManager دوره‌ای با constraint
آپلود ۲۰۰ عکسی که کاربر همین الان انتخاب کرده	UIDT در اندروید ۱۴ به بعد؛ پایین‌تر، worker طولانی‌مدت
پخش موسیقی	`MediaSessionService` در Media3 (نوع `mediaPlayback`)
ثبت دویدن، مسیریابی لحظه‌به‌لحظه	foreground service از نوع `location`
زنگ بیدارباش ساعت ۷ صبح	`setAlarmClock()` با `USE_EXACT_ALARM`
سفارش جدیدی روی سرور منتظر است	FCM با اولویت بالا، سپس notification

تلهٔ مصاحبه

«WorkManager برای همه چیز» و *«foreground service برای همه چیز»* هر دو اشتباه‌اند. اولی کار قابل‌مشاهده برای کاربر را خراب می‌کند؛ دومی در بررسی Play رد می‌شود و باتری را خالی می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. مهاجرت یک اپ قدیمی به targetSdk 35 به بالا

حوزه	چه باید کرد
هر FGS	نوع و مجوز نوع در manifest؛ نوع در `startForeground()` ؛ اعلام در Play Console
سرویس‌های `dataSync`	پیاده‌سازی `onTimeout()` ؛ بخش‌های قابل‌تعویق به WorkManager، انتقال‌های کاربر به UIDT
شروع از پس‌زمینه	هر `startForegroundService()` را در receiverها، pushها و workerها پیدا کنید؛ catch و جایگزین
boot receiverها	هیچ FGS از نوع `dataSync` ، دوربین، میکروفون، رسانه یا تماس از `BOOT_COMPLETED`
exact alarmها	`canScheduleExactAlarms()` به‌علاوهٔ جایگزین غیردقیق؛ `USE_EXACT_ALARM` را توجیه یا حذف کنید
notificationها	درخواست `POST_NOTIFICATIONS` در لحظه‌ای که منطقی است
jobها	تست در bucket محدودشده و در Doze؛ quotaهای زمان اجرای سخت‌گیرانه‌تر در اندروید ۱۶

نظر من

به ترتیبی مهاجرت کنید که چیزها **crash می‌کنند**: اول نوع FGS و شروع از پس‌زمینه (exception)، بعد رفتار boot و alarm (خطاهای بی‌صدا)، بعد بهینه‌سازی باتری. بالا بردن targetSdk را با staged rollout منتشر کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. رگرسیون‌ها را قابل‌مشاهده کنید

- خطاهای `ForegroundServiceStartNotAllowedException` و `MissingForegroundServiceTypeException` و `SecurityException` ناشی از `startForeground()` را به شکل خطای **non-fatal** ثبت کنید، با تگ نسخهٔ اندروید
- ماتریس تست روی امولاتورهای API 31 و 34 و 35 و 36 — شروع از پس‌زمینه، ری‌استارت، لغو مجوز exact alarm، رد کردن notification
- دستورهای مفید:

```
adb shell dumpsys activity services com.example.app
adb shell dumpsys alarm | grep com.example.app
adb shell am set-standby-bucket com.example.app restricted
```

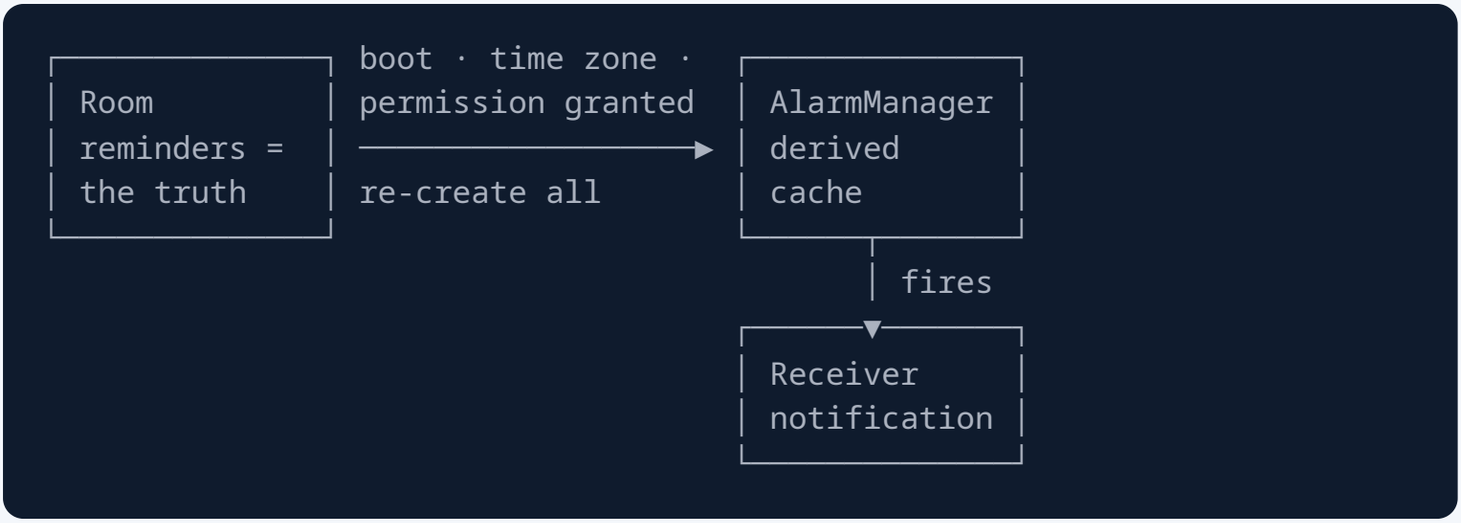
تلهٔ مصاحبه

«targetSdk را بالا ببر، build را درست کن، منتشر کن.» — بیشتر این تغییرها build را نمی‌شکنند؛ در زمان اجرا روی نسخه‌های خاصی exception می‌دهند، یا بی‌صدا دیگر اجرا نمی‌شوند. بدون آن ماتریس تست، کاربرها اول پیدایشان می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. طراحی: یادآورهایی که واقعاً اجرا می‌شوند



- Room **منبع حقیقت** است؛ alarmها یک cache مشتق‌شده‌اند که هر لحظه می‌شود دوباره ساختشان — به شکل idempotent
- دوباره‌سازی در این موقعیت‌ها: boot، `ACTION_TIMEZONE_CHANGED` و `ACTION_TIME_CHANGED` (هر دو هنوز برای receiverهای manifest مجازند)، اعطای مجوز exact alarm، و شروع اپ
- اگر مجاز است exact alarm؛ وگرنه alarm غیردقیق از نوع while-idle **و به کاربر بگویید** یادآورها ممکن است دیر برسند، با لینکی به تنظیمات

تلهٔ مصاحبه

«*alarm را یک بار، وقتی کاربر یادآور را می‌سازد، زمان‌بندی کن.*» — ری‌استارت یا force stop آن را پاک می‌کند، و تغییر منطقهٔ زمانی معنای «ساعت ۸» را عوض می‌کند. بدون دوباره‌سازی از منبع حقیقت، یادآورها بی‌صدا متوقف می‌شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. چه چیزی آن را قابل‌اعتماد می‌کند

```
class ReminderReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        val id = intent.getLongExtra(EXTRA_REMINDER_ID, -1)
        val title = intent.getStringExtra(EXTRA_TITLE)
        if (id < 0 || title == null) return
        val nm = NotificationManagerCompat.from(context)
        if (nm.areNotificationsEnabled()) {
            ReminderNotifier(context).show(id, title)
        }
        // Next occurrence: read Room off the main thread
        RescheduleWorker.enqueue(context, id)
    }
}
```

- `PendingIntent` با `FLAG_IMMUTABLE` و یک request code ثابت برای هر یادآور، تا زمان‌بندی دوباره **جایگزین** شود نه تکراری
- UX صادقانه: اگر notificationها خاموش است، exact alarm رد شده یا اپ force stop شده، به‌جای شکست بی‌صدا آن را در اپ **نشان دهید**

جواب سینیور در یک جمله

«سبک‌ترین قرارداد با سیستم را انتخاب می‌کنم که نیاز را برآورده کند — و برای لحظه‌ای طراحی می‌کنم که سیستم می‌گوید نه: جایگزین‌ها، زمان‌بندی دوباره از منبع حقیقت، و UI صادق.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۶ — خلاصهٔ یک‌صفحه‌ای

- callbackهای Service روی **ترد اصلی** اجرا می‌شوند؛ FGS با process خودش می‌میرد.
- سرویس پس‌زمینه از پس‌زمینه شروع نمی‌شود (اندروید ۸٫۰) و زود متوقف می‌شود.
- bound service تا وقتی کلاینت‌ها bind هستند زنده است؛ `onServiceDisconnected()` یعنی crash، نه unbind.
- هر FGS یعنی notification، **نوع** و مجوز نوع (اندروید ۱۴)؛ `startForeground()` سریع.
- شروع FGS از پس‌زمینه (اندروید ۱۲) جز در استثناها ممنوع است — catch و جایگزین.
- FGS از نوع `dataSync`: ۶ ساعت در هر ۲۴ ساعت (اندروید ۱۵) — `onTimeout()` را پیاده کنید.
- exact alarm یعنی `SCHEDULE_EXACT_ALARM` (از ۱۴ پیش‌فرض رد) یا `USE_EXACT_ALARM` (ساعت و تقویم).
- UIDT برای انتقال حجیم به درخواست کاربر؛ job با `setPersisted(true)` از ری‌استارت عبور می‌کند، alarm نه.
- FCM با اولویت بالا فقط برای چیزی که کاربر می‌بیند — وگرنه تنزل پیدا می‌کند.
- آرکیتکت: اول سبک‌ترین سازوکار، یک منبع حقیقت برای دوباره‌سازی، و یک جایگزین برای هر «نه».

روز ۱۷ در راه است: کارایی، بخش ۱ — رندرینگ، jank، حافظه و profiling. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700